

```

#include <iostream>
using namespace std;

class Node {

public:
    int day;
    char status;
    Node* next;
    Node(int d, char s) {
        day = d;
        status = s;
        next = nullptr;
    }

    int getDay() const { return day; }
    char getStatus() const { return status; }
    void setStatus(char s) { status = s; }
    Node* getNext() const { return next; }
    void setNext(Node* n) { next = n; }
};

class AttendanceList {
private:
    Node* head;
    int dayCounter;

public:
    AttendanceList() {
        head = nullptr;
        dayCounter = 0;
    }

    void insertAttendance(char status) {
        if (status == 'p') status = 'P';
        else if (status == 'a') status = 'A';

        if (status != 'P' && status != 'A') {
            cout << "Invalid status! Please enter 'P' or 'A'";
        }

        dayCounter++;
        Node* newNode = new Node(dayCounter, status);

        if (head == nullptr) {
            head = newNode;
        } else {
            Node* temp = head;
            while (temp->getNext() != nullptr) {
                temp = temp->getNext();
            }
            temp->setNext(newNode);
        }
    }
};

```

```

        cout << "Attendance recorded for Day " << dayCounter << ": " <<
status << "\n";
    }

```

```

void updateAttendance(int targetDay, char newStatus) {
    if (newStatus == 'p') newStatus = 'P';
    else if (newStatus == 'a') newStatus = 'A';

    if (newStatus != 'P' && newStatus != 'A') {
        cout << "Invalid status! Please enter 'P' or 'A'. \n";
        return;
    }

    Node* temp = head;
    while (temp != nullptr) {
        if (temp->getDay() == targetDay) {
            temp->setStatus(newStatus);
            cout << "Attendance for Day " << targetDay << "
successfully updated to " << newStatus << ". \n";
            return;
        }
        temp = temp->getNext();
    }
    cout << "Day " << targetDay << " not found. \n";
}

```

```

void calculateTotals() {
    int presentCount = 0;
    int absentCount = 0;
    Node* temp = head;

    while (temp != nullptr) {
        if (temp->getStatus() == 'P') {
            presentCount++;
        } else if (temp->getStatus() == 'A') {
            absentCount++;
        }
        temp = temp->getNext();
    }

    cout << "\n--- Attendance Summary ---\n";
    cout << "Total Present Days : " << presentCount << "\n";
    cout << "Total Absent Days : " << absentCount << "\n";
    cout << "-----\n";
}

```

```

void displayAttendance() {
    if (head == nullptr) {
        cout << "No attendance records found. \n";
        return;
    }

    cout << "\n--- Complete Attendance Record ---\n";
    cout << "Day\t|\tStatus\n";

```

```

        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->getDay() << "\t|\t" << (temp->getStatus() ==
'P' ? "Present (P)" : "Absent (A)") << "\n";
            temp = temp->getNext();
        }
    }

    ~AttendanceList() {
        Node* current = head;
        while (current != nullptr) {
            Node* nextNode = current->getNext();
            delete current;
            current = nextNode;
        }
    }
};

int main() {
    AttendanceList studentRecord;
    int choice, day;
    char status;

    do {
        cout << "\n=== Student Attendance Manage System ===\n";
        cout << "1. Insert Attendance (P/A)\n";
        cout << "2. Update Attendance for a Day\n";
        cout << "3. Calculate Total Present/Absent Days\n";
        cout << "4. Display Complete Attendance List\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter attendance (P - Present, A - Absent): ";
                cin >> status;
                studentRecord.insertAttendance(status);
                break;
            case 2:
                cout << "Enter day number to update: ";
                cin >> day;
                cout << "Enter new status (P/A): ";
                cin >> status;
                studentRecord.updateAttendance(day, status);
                break;
            case 3:
                studentRecord.calculateTotals();
                break;
            case 4:
                studentRecord.displayAttendance();
                break;
            case 5:
                cout << "Program Exited Successfully.\n";

```

```
        break;
    default:
        cout << "Invalid choice!\n";
    }
} while (choice != 5);

return 0;
}
```